

Modulo3_Analisis_Series_Temporales

June 30, 2026

1 Módulo 3: Análisis de Series Temporales

Data Analytics con Pronósticos a través de Python

Duración: 4 horas

```
[6]: pip install statsmodels pmdarima
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting statsmodels
  Downloading statsmodels-0.14.6-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(9.5 kB)
Collecting pmdarima
  Downloading pmdarima-2.1.1-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(8.5 kB)
Requirement already satisfied: numpy<3,>=1.22.3 in ~/.local/lib/python3.10/site-
packages (from statsmodels) (2.2.6)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in
./~/.local/lib/python3.10/site-packages (from statsmodels) (1.15.1)
Requirement already satisfied: pandas!=2.1.0,>=1.4 in
./~/.local/lib/python3.10/site-packages (from statsmodels) (2.2.3)
Collecting patsy>=0.5.6 (from statsmodels)
  Downloading patsy-1.0.2-py2.py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: packaging>=21.3 in ~/.local/lib/python3.10/site-
packages (from statsmodels) (25.0)
Requirement already satisfied: joblib>=0.11 in ~/.local/lib/python3.10/site-
packages (from pmdarima) (1.4.2)
Collecting Cython!=0.29.18,!0.29.31,>=0.29 (from pmdarima)
  Downloading cython-3.2.8-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
(4.2 kB)
Requirement already satisfied: scikit-learn>=0.22 in
./~/.local/lib/python3.10/site-packages (from pmdarima) (1.6.1)
Requirement already satisfied: urllib3 in /usr/lib/python3/dist-packages (from
pmdarima) (1.26.5)
Requirement already satisfied: setuptools!=50.0.0,>=42 in /usr/lib/python3/dist-
packages (from pmdarima) (59.6.0)
```

```

Requirement already satisfied: python-dateutil>=2.8.2 in
./..local/lib/python3.10/site-packages (from pandas!=2.1.0,>=1.4->statsmodels)
(2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./..local/lib/python3.10/site-
packages (from pandas!=2.1.0,>=1.4->statsmodels) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in ./..local/lib/python3.10/site-
packages (from pandas!=2.1.0,>=1.4->statsmodels) (2024.2)
Requirement already satisfied: six>=1.5 in ./..local/lib/python3.10/site-packages
(from python-dateutil>=2.8.2->pandas!=2.1.0,>=1.4->statsmodels) (1.17.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
./..local/lib/python3.10/site-packages (from scikit-learn>=0.22->pmdarima)
(3.5.0)
Downloading statsmodels-0.14.6-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (10.4 MB)

10.4/10.4 MB 8.6 MB/s 0:00:01a
0:00:0136m0:00:01
Downloading pmdarima-2.1.1-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (694 kB)

694.8/694.8 kB 7.8 MB/s 0:00:00
Downloading cython-3.2.8-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.5 MB)

3.5/3.5 MB 10.3 MB/s 0:00:002.3 MB/s eta
0:00:01
Downloading patsy-1.0.2-py2.py3-none-any.whl (233 kB)
WARNING: Error parsing dependencies of python-apt: Invalid version:
'2.4.0-ubuntu4.1-zorin1'

Installing collected packages: patsy, Cython, statsmodels, pmdarima

4/4 [pmdarima] 2/4 [statsmodels]
Successfully installed Cython-3.2.8 patsy-1.0.2 pmdarima-2.1.1
statsmodels-0.14.6

[notice] A new release of pip is
available: 26.0.1 -> 26.1.2
[notice] To update, run:
python3 -m pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.

```

```

[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose

```

```

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA

%matplotlib inline
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

```

1.1 1. Carga de Datos y Preparación

```

[3]: # Crear dataset de ejemplo de serie temporal
np.random.seed(42)
dates = pd.date_range(start='2023-01-01', periods=365, freq='D')

data = {
    'fecha': dates,
    'ventas': 1200 + np.sin(np.arange(365) * 2 * np.pi / 30) * 300 + np.random.
        ↪normal(0, 80, 365) + np.arange(365) * 2
}

df = pd.DataFrame(data)
df.set_index('fecha', inplace=True)
df.head()

```

```

[3]:
      fecha      ventas
2023-01-01  1239.737132
2023-01-02  1253.312363
2023-01-03  1377.836076
2023-01-04  1504.177964
2023-01-05  1412.211178

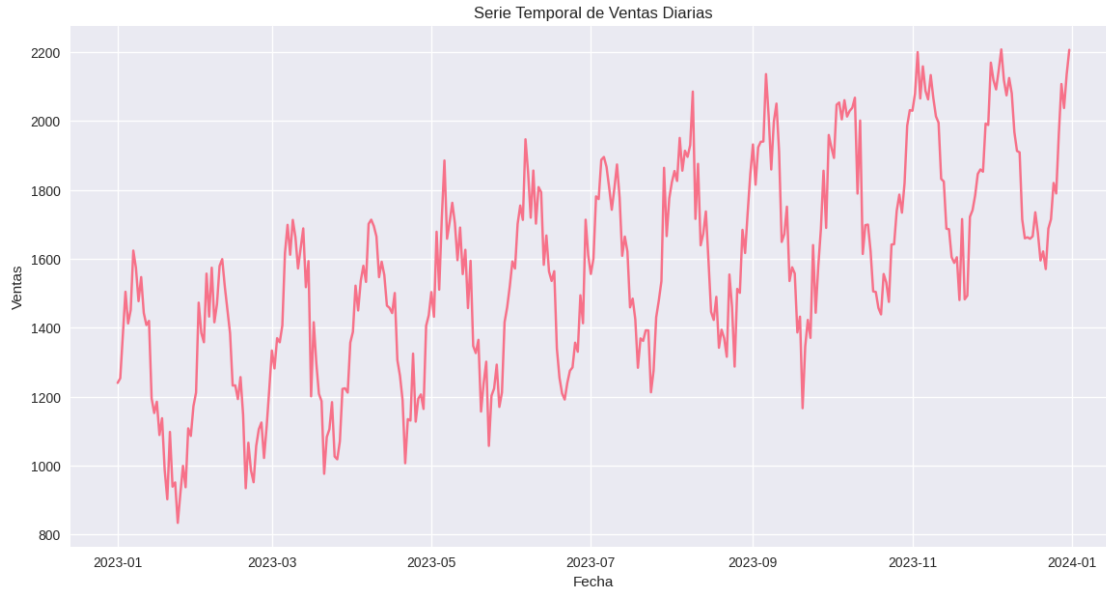
```

1.2 2. Análisis Exploratorio de Series Temporales

```

[4]: plt.figure(figsize=(14,7))
plt.plot(df['ventas'])
plt.title('Serie Temporal de Ventas Diarias')
plt.xlabel('Fecha')
plt.ylabel('Ventas')
plt.grid(True)
plt.show()

```



1.3 3. Descomposición de la Serie Temporal

```
[16]: def plot_decomposition_clean(serie, period=30):
    decomposition = seasonal_decompose(serie, model='additive', period=period)

    fig = decomposition.plot()
    fig.set_size_inches(15, 10)

    fig.suptitle('Descomposición Completa de la Serie Temporal',
                 fontsize=20,
                 fontweight='bold',
                 y=1.02)

    # Títulos más claros
    fig.axes[0].set_title('Serie Original', fontsize=14, fontweight='bold')
    fig.axes[1].set_title('Tendencia', fontsize=14, fontweight='bold')
    fig.axes[2].set_title('Componente Estacional', fontsize=14,
    ↪fontweight='bold')
    fig.axes[3].set_title('Residuos', fontsize=14, fontweight='bold')

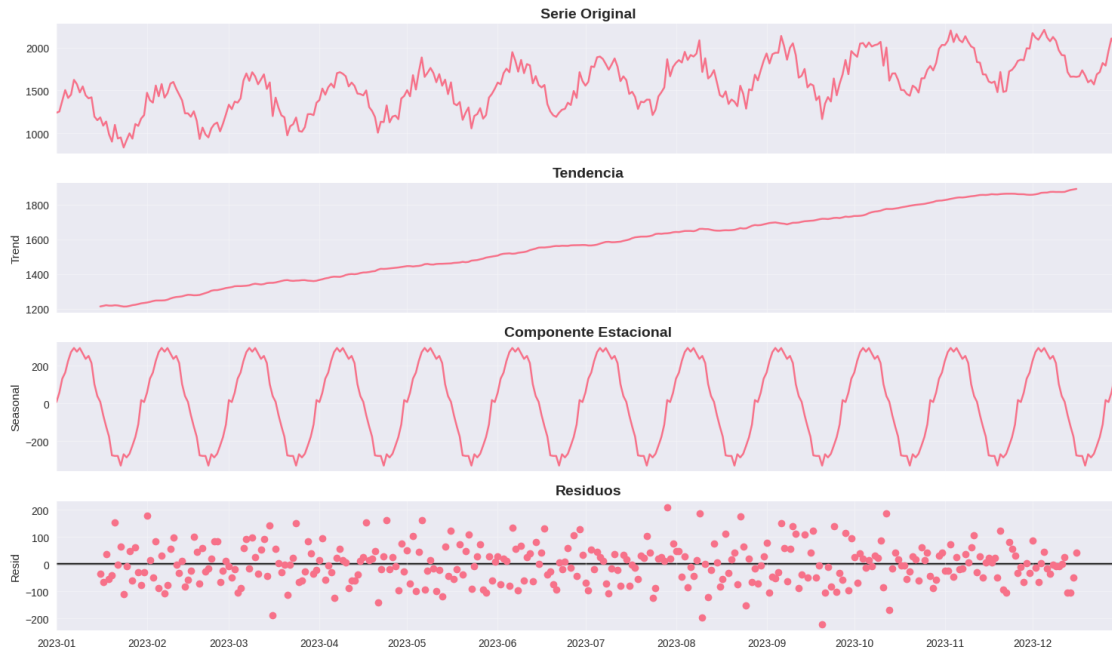
    for ax in fig.axes:
        ax.grid(True, alpha=0.35)

    plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Espacio superior e inferior
    plt.show()

    # Usar la función:
```

```
plot_decomposition_clean(df['ventas'], period=30)
```

Descomposición Completa de la Serie Temporal



1.4 4. Prueba de Estacionariedad (ADF Test)

```
[17]: def prueba_estacionariedad(serie):
      result = adfuller(serie)
      print('ADF Statistic:', result[0])
      print('p-value:', result[1])
      print('Valores críticos:', result[4])

      prueba_estacionariedad(df['ventas'])
```

```
ADF Statistic: -0.811911068874121
p-value: 0.8155982184915758
Valores críticos: {'1%': np.float64(-3.449336554273722), '5%':
np.float64(-2.8699055166063085), '10%': np.float64(-2.571226758215748)}
```

1.5 5. Modelos de Pronóstico

```
[21]: # Preparación del índice con frecuencia
      df = df.asfreq('D') # Asegura frecuencia diaria

      print("Frecuencia del índice:", df.index.freq)
```

```

# Suavizado Exponencial Holt-Winters
model_hw = ExponentialSmoothing(
    df['ventas'],
    trend='add',
    seasonal='add',
    seasonal_periods=30
)

fit_hw = model_hw.fit()
pronostico_hw = fit_hw.forecast(60)

print(" Pronóstico Holt-Winters generado para los próximos 60 días")

```

Frecuencia del índice: <Day>

Pronóstico Holt-Winters generado para los próximos 60 días

```

[22]: # ARIMA
model_arima = ARIMA(df['ventas'], order=(1,1,1))
fit_arima = model_arima.fit()
pronostico_arima = fit_arima.forecast(steps=60)

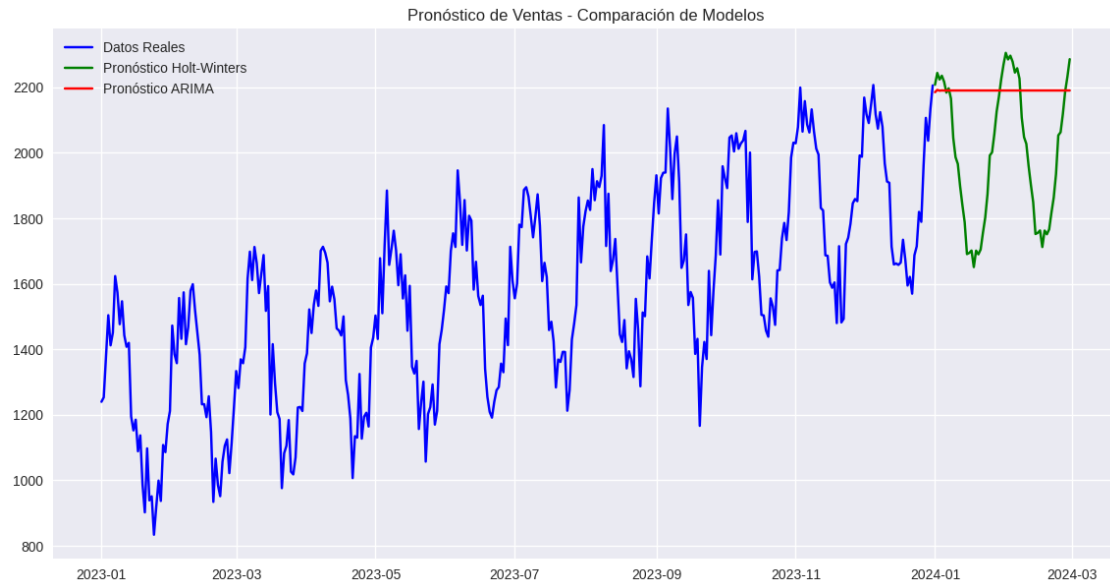
```

1.6 6. Visualización de Pronósticos

```

[23]: plt.figure(figsize=(14,7))
plt.plot(df.index, df['ventas'], label='Datos Reales', color='blue')
plt.plot(pd.date_range(start=df.index[-1], periods=61, freq='D')[1:],
    ↪pronostico_hw, label='Pronóstico Holt-Winters', color='green')
plt.plot(pd.date_range(start=df.index[-1], periods=61, freq='D')[1:],
    ↪pronostico_arima, label='Pronóstico ARIMA', color='red')
plt.title('Pronóstico de Ventas - Comparación de Modelos')
plt.legend()
plt.grid(True)
plt.show()

```



¡Fin del Módulo 3!

Tarea: Genera pronósticos con ambos modelos y compara su desempeño usando MAE y RMSE.